

SEC Filing Intelligence System

*An Orchestrated RAG Framework for Financial Analysis
using SEC EDGAR Filings*

B.Tech Final Year Project Presentation

Soham Dutta | Rajat Sharma

Department of Computer Science & Engineering | 2024–25

Problem Statement

What's broken with SEC filings today?

- SEC filings — 10-K, 10-Q, 8-K — run into hundreds of pages. A single 10-K can exceed 150 pages of dense financial language.
- They are unstructured: risk factors, MD&A sections, footnotes, and exhibits are scattered across the document without any consistent machine-readable format.
- Analysts have to manually scan these documents, cross-reference multiple filings, and piece together a coherent view — a process that takes days.
- Existing keyword search tools return raw text snippets with no analytical depth. They tell you WHERE a phrase appears, not WHAT it means.

150+

pages per 10-K filing

3 types

of filings to cross-reference

Days

of analyst effort saved

Motivation

Why does this problem matter in the real world?

- Every quarter, thousands of institutional investors, analysts, and retail traders need to quickly understand how a company is performing — not just in isolation, but across time and across filing types.
- Investment decisions, risk assessments, and earnings calls all depend on the kind of insights buried inside these filings. But the speed of markets doesn't wait for manual reading.
- The demand for automated, AI-powered financial intelligence is growing rapidly. Yet most existing tools either extract raw text (no analysis) or require expensive data vendors with proprietary datasets.
- This project was motivated by a very practical gap: build an open, accessible system that can take any company's public SEC filings and produce structured, analyst-quality insights — without human intervention.
- Regulatory compliance teams, financial journalists, startup investors, and even students doing equity research can all benefit from a system that bridges raw legal filings and actionable financial intelligence.

Real Example: Why All Three Filings Matter

Imagine a query: 'Assess Microsoft's financial health in 2023.'

10-K

- Annual report — covers full-year revenue, operating income, segment performance
- Long-term strategic direction, risk factors, competitive landscape
- Gives the 'big picture' of where the company stands

10-Q

- Quarterly report — covers recent 3-month performance
- Shows how current quarter deviates from annual trend
- Useful for detecting momentum shifts mid-year

8-K (Future)

- Event-triggered filing — covers material events in real-time (planned for future scope)
- Covers earnings surprises, leadership changes, acquisitions
- Gives the 'what just happened' context missing in 10-K/Q

This project uses 10-K and 10-Q filings; 8-K support is identified as future scope.

Literature Review

Reviewing foundational research that shaped this project

Paper 1A: Lewis et al., 2020

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks — Patrick Lewis, Ethan Perez, et al.

What They Did:

- Lewis et al. proposed a novel architecture that combines a dense retrieval component with a generative language model, allowing the model to retrieve relevant documents before generating answers.
- The retrieval component uses a Dense Passage Retriever (DPR) that encodes both queries and documents into a shared embedding space, enabling semantic (not just keyword) search.
- The generative component is a seq2seq model (BART) that conditions its output on both the query and the retrieved documents, producing fluent, grounded responses.
- The model was evaluated on open-domain question answering benchmarks like Natural Questions, TriviaQA, and WebQuestions — outperforming previous closed-book LM baselines by significant margins.
- This work effectively introduced the RAG paradigm — the foundation on which this project's retrieval pipeline is built.

Paper 1B: Techniques, Results & Limitations

Ref: Lewis et al., 2020

Techniques & Results

- Dense Passage Retrieval (DPR): bi-encoder architecture encoding queries and passages independently using BERT
- BART as the generator: conditioned on [query + top-k retrieved passages]
- Two variants: RAG-Sequence (retrieve once per output) and RAG-Token (retrieve at each generation step)
- Outperformed T5 (closed-book) on NQ by ~10 F1 points
- State-of-the-art on Open MS-MARCO at the time of publication



Limitations

- Designed for open-domain QA over Wikipedia — not multi-document financial corpora with varied structure
- No orchestration layer: retrieval is a single flat step, not query-conditioned or decomposed
- Struggles with multi-hop reasoning — retrieving one document at a time limits cross-document synthesis
- Does not handle domain-specific document formats (10-K sections, MD&A, exhibits) natively
- No structured output — results are free-form text, unsuitable for analytical reporting

Paper 2A: Chen et al., 2021

FinQA: A Dataset of Numerical Reasoning over Financial Data — Zhiyu Chen, Wenhui Chen, et al.

What They Did:

- Chen et al. introduced FinQA — a large-scale dataset of question-answer pairs that require both textual and numerical reasoning over real financial reports (10-K filings from S&P 500 companies).
- The dataset includes 8,281 QA pairs requiring multi-step calculations such as percentage changes, year-over-year comparisons, and ratio computations drawn from financial tables and paragraphs.
- They proposed a two-stage pipeline: a retrieval module that identifies relevant text/table segments, and an NLP model that generates a structured program (not just a text answer) to compute the final result.
- The program generation approach represents financial reasoning as executable operations (e.g., subtract, divide, percentage) applied to retrieved values — a key step toward verifiable reasoning.
- FinQA was one of the first benchmarks to treat financial documents as a structured reasoning challenge rather than a simple information extraction task.

Paper 2B: Techniques, Results & Limitations

Ref: Chen et al., 2021

Techniques & Results

- Retrieval: a BERT-based retriever selects the most relevant table rows and text passages from the report
- Program generation: a seq2seq model (RoBERTa + LSTM decoder) generates arithmetic programs to derive numerical answers
- Evaluation metric: execution accuracy — whether the generated program produces the correct final number
- Baseline model achieved ~61% execution accuracy on the test set
- Human performance was ~91% — indicating significant room for improvement in machine reasoning



Limitations

- Focused exclusively on numerical reasoning — cannot handle qualitative or strategic analytical questions
- Requires structured table data; fails when financial info is embedded in narrative prose
- The dataset covers only earnings reports (10-K) — no 8-K material events or 10-Q quarterly filings
- The program generation approach is brittle — slight phrasing changes in questions can break the pipeline
- No support for multi-document or cross-filing reasoning

Paper 3A: Araci, 2019

FinBERT: Financial Sentiment Analysis with Pre-trained Language Models — Dogu Araci

What They Did:

- Araci proposed FinBERT — a domain-adapted version of BERT specifically fine-tuned on financial text, addressing the mismatch between general-purpose BERT and the specialized vocabulary of finance.
- The pre-training corpus included financial news articles, earnings call transcripts, and analyst reports — domain-specific text that BERT's original Wikipedia/BookCorpus training had never seen.
- FinBERT was then fine-tuned for financial sentiment classification, categorizing text segments as Positive, Negative, or Neutral with respect to financial outcomes.
- The model showed strong performance on the Financial PhraseBank dataset, significantly outperforming vanilla BERT on financial sentiment classification.
- This work established that domain-specific pre-training matters greatly in finance — a principle that informed how we think about embedding quality in this project.

Paper 3B: Techniques, Results & Limitations

Ref: Araci, 2019

Techniques & Results

- Architecture: BERT-base architecture with financial domain-specific pre-training on 4.9B tokens of financial text
- Fine-tuning: softmax classifier on [CLS] token for 3-class sentiment output
- Dataset: Financial PhraseBank (4,840 sentences manually labeled by domain experts)
- Achieved ~97% accuracy on Financial PhraseBank (all-agreement sentences)
- Outperformed vanilla BERT by ~5–8% on financial sentiment tasks



Limitations

- Designed purely for sentiment classification — not for semantic similarity or retrieval tasks
- Cannot be used for dense passage retrieval or embedding-based search (not a bi-encoder)
- Trained on news/analyst text, not SEC filings specifically — MD&A and risk factor sections differ stylistically
- Binary/ternary sentiment is too coarse for analytical reasoning over multi-section documents
- Does not capture cross-sentence or cross-paragraph relationships needed for report-level understanding

Paper 4A: Malo et al.

Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts — Malo, Sinha, Korhonen, Wallenius, Takala

What They Did:

- Malo et al. addressed the challenge of sentiment analysis in economic and financial text, where the same phrase can have opposite implications depending on context (e.g., 'interest rates fell' is good for borrowers, bad for banks).
- The authors created the Financial PhraseBank — a carefully curated dataset of 4,840 sentences from financial news, annotated by 16 domain experts for sentiment (positive, negative, neutral).
- They proposed context-aware methods for financial sentiment that go beyond surface-level polarity, incorporating world knowledge about economic relationships.
- The work highlighted the importance of domain expertise in annotation — sentences where annotators disagreed were categorized separately, enabling granular evaluation.
- Their dataset has become the standard benchmark for financial sentiment analysis, used to evaluate FinBERT, and subsequent financial NLP models.

Paper 4B: Techniques, Results & Limitations

Ref: Malo et al.

Techniques & Results

- Manual annotation by 16 financial domain experts across 4,840 sentences
- Sentence-level sentiment with context-aware disambiguation for economic polarity
- Used SVM and naive Bayes classifiers as baselines with financial lexicons
- Agreement subsets showed ~75–78% accuracy using classical ML; deep learning models (FinBERT) later reached ~97%
- Dataset became the de facto standard benchmark in financial NLP



Limitations

- Focuses only on news sentences — SEC filing language (legal disclaimers, accounting footnotes) has different characteristics
- Sentence-level annotations miss document-level context: a negative sentence in a boilerplate risk section means something very different from one in the MD&A
- Classical ML classifiers used as baselines are now significantly outperformed
- No framework for connecting sentiment signals to structured financial metrics or retrieval
- Does not scale to full-document analysis across multiple filing types

Paper 5A: Reimers & Gurevych, 2019

Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks — Nils Reimers, Iryna Gurevych

What They Did:

- Reimers and Gurevych addressed a core problem with BERT: generating meaningful sentence-level embeddings at scale. BERT's cross-encoder architecture requires comparing every sentence pair jointly, making large-scale similarity search computationally infeasible.
- They proposed Sentence-BERT (SBERT), a modification of BERT that uses siamese and triplet network structures to produce fixed-size sentence embeddings that can be compared using cosine similarity.
- SBERT significantly reduces the time needed for semantic similarity computations — from 65 hours with BERT (for 10K sentence pairs) to about 5 seconds with SBERT — enabling practical large-scale retrieval.
- The model was fine-tuned on NLI and STS datasets using a softmax/cosine similarity objective, learning to pull semantically similar sentences closer in embedding space.
- This work directly enables the embedding-based retrieval layer in our project — we use Sentence Transformers (the library built on SBERT) to embed both query and document chunks.

Paper 5B: Techniques, Results & Limitations

Ref: Reimers & Gurevych, 2019

Techniques & Results

- Siamese network: two identical BERT encoders share weights; mean-pooling on token embeddings produces a fixed 768-dim sentence vector
- Training objective: softmax loss on NLI (entailment/contradiction/neutral), and cosine similarity regression on STS-B
- Evaluation on Semantic Textual Similarity (STS) benchmarks and downstream IR tasks
- Achieved ~86.9 Spearman correlation on STS-B — significantly outperforming averaged BERT embeddings (53.2)
- Order-of-magnitude speedup over BERT cross-encoder for large-scale retrieval



Limitations

- General-purpose model — not fine-tuned on financial text, which has distinct vocabulary and sentence structure
- Fixed-size embeddings may lose fine-grained local information in long paragraphs (>512 tokens)
- Does not inherently understand numerical relationships or financial metrics
- No built-in mechanism for query decomposition or multi-document reasoning
- Embedding quality degrades for very domain-specific technical language without domain adaptation

Paper 6A: Yao et al., 2023

ReAct: Synergizing Reasoning and Acting in Language Models — Shunyu Yao, Jeffrey Zhao, et al.

What They Did:

- Yao et al. proposed ReAct — a framework that interleaves chain-of-thought reasoning with action steps (such as tool calls or database lookups) in a single LLM inference loop.
- The key insight is that reasoning and acting should not be separate: an LLM should be able to think about what it needs (reasoning), take an action to retrieve it (acting), observe the result, and continue reasoning — in a tight loop.
- ReAct agents can search Wikipedia, query databases, or call external APIs mid-reasoning, making them significantly more accurate on tasks requiring information that isn't in the model's weights.
- They demonstrated ReAct on HotpotQA (multi-hop reasoning), FEVER (fact verification), and ALFWorld (interactive game environments) — showing strong improvements over chain-of-thought-only baselines.
- ReAct directly inspired the agentic orchestration philosophy in this project, though our system applies it in a more structured, financial-domain-specific manner.

Paper 6B: Techniques, Results & Limitations

Ref: Yao et al., 2023

Techniques & Results

- Framework: alternating Thought → Action → Observation cycles within a single prompt context
- Tools: Wikipedia search API, internal lookup API for fact verification tasks
- Models: GPT-3 (text-davinci-002) and PaLM with few-shot prompting
- On HotpotQA: ReAct outperformed chain-of-thought by ~6% Exact Match
- On FEVER: reduced hallucination rate significantly compared to CoT-only baselines

Limitations

- Reasoning loop is free-form and unstructured — each action is decided ad hoc without a pre-designed orchestration plan
- For financial analysis, ad-hoc retrieval is insufficient: specific filing types must be fetched in a deliberate, intent-driven sequence
- ReAct doesn't decompose queries into multiple parallel analytical sub-questions — it handles one thread at a time
- No structured output format — outputs are conversational, not reportable analytical sections
- Latency grows linearly with reasoning steps — not optimized for parallel multi-document retrieval

Literature Review: Comparison Table

Paper	Methodology	Dataset	Key Strength	Key Limitation
Lewis et al. (RAG)	Dense retrieval + generative model	NQ, TriviaQA	End-to-end retrieval + generation	Single-step, no orchestration
Chen et al. (FinQA)	Retriever + program generator	S&P 500 10-Ks	Numerical reasoning support	No qualitative analysis or 8-K/10-Q
Araci (FinBERT)	Domain pre-train + fine-tune	Financial PhraseBank	Strong financial sentiment accuracy	Not usable for retrieval/embedding
Malo et al.	Expert annotation + classical ML	Custom news corpus	High-quality financial labeled data	Sentence-level only, no document analysis
Reimers & Gurevych (SBERT)	Siamese BERT for sentence embeddings	NLI + STS-B	Fast, scalable semantic search	General-purpose, not finance-adapted
Yao et al. (ReAct)	Interleaved reasoning + action steps	HotpotQA, FEVER	Dynamic tool-use during reasoning	Ad-hoc, no structured multi-doc analysis

Research Gap

What existing systems still cannot do:

✗ No cross-filing analysis

Current systems handle one document type at a time. None can simultaneously reason across 10-K (annual), 10-Q (quarterly), and 8-K (event) filings for a single query.

✗ No query decomposition for financial intent

Queries like 'Assess Amazon's risk profile' require multiple sub-questions mapped to different filing sections. Existing RAG systems treat this as one flat retrieval.

✗ No structured analytical output

Existing tools return raw snippets or free-form text. No system produces a structured, section-wise analytical report directly from unstructured filings.

✗ No orchestrated parallel processing

ReAct and RAG process queries sequentially. Financial analysis benefits enormously from parallel retrieval across filing types — a capability absent in current work.

This project directly addresses all four gaps through its orchestrated, multi-step RAG architecture.

Project Objectives

01

Query Understanding

Parse a natural language financial query into structured fields: company ticker, date range, intent class (full_report or specific), and filing types — validated via Pydantic schema using llama-3.1-8b-instant.

02

Targeted Data Retrieval

Fetch the right filings from SEC EDGAR based on parsed intent — 10-K for fundamentals, 10-Q for recent trends, 8-K for material events — not all filings indiscriminately.

03

Query Decomposition

Break the original query into three parallel analytical sub-queries, each aligned with a specific filing type and a distinct dimension of financial analysis.

04

Parallel Reasoning

Fan out parallel llm_call workers via LangGraph's Send API for full-report queries, with each worker retrieving k=5 chunks filtered by filing type and section metadata.

05

Structured Output

Synthesize all retrieved insights into a clean, section-wise analytical report — combining MD&A, risk factors, financial statements, and business factors from 10-K and 10-Q filings.

Proposed System Overview

This is not a standard RAG system.

- A standard RAG system takes a query, converts it to an embedding, fetches the top-k similar chunks, and hands them to an LLM for a single-pass answer. That's retrieval augmentation — useful, but shallow.
- Our system does something fundamentally different: it first understands the intent of the query, then decides which filing types to use, decomposes the query into targeted sub-questions, runs parallel retrieval for each, applies LLM reasoning per sub-section, and finally synthesizes everything into a structured analytical report.
- The orchestration layer is the core innovation. It acts as a conductor — it knows which instrument (filing type) to involve, when, and for what analytical purpose.
- Using LangGraph as the orchestration framework, the pipeline is a directed state graph: `query_decomposer` → `constructDB` → `intent_router` → (`retriever` + `llm_response`) OR (`orchestrator` → `parallel_llm_call_workers` → `synthesizer`).
- The result is a system where the quality of output scales with the

System Flow

1. Intent Parsing

2. EDGAR Data Fetch

3. Vector DB Construction

4. Intent Routing

5. Parallel LLM Workers

6. Report Synthesis

Methodology (1/5): Query Parsing & Intent Extraction

Step 1: Understanding what the user really wants

- The system first passes the raw user query through a structured LLM call using llama-3.1-8b-instant (via Groq). This parses the query into five typed fields: company ticker, start date, optional end date, intent class (full_report or specific), and the filing types to retrieve.
- For example, a query like “Provide a financial report on MSFT” is parsed into: {ticker: MSFT, start_date: ..., intent: full_report, filing_types: [10-K, 10-Q]}. The output is validated against a Pydantic schema before any downstream node consumes it.
- This validated structured object drives everything downstream — it determines which filings to fetch from EDGAR, whether to route to the specific query workflow or the full-report orchestrator, and what retrieval filters to apply in ChromaDB.
- Pydantic validation is a key design choice here. If the model returns a response that fails the QueryDecompose schema, LangChain raises an error rather than passing malformed state downstream — preventing silent failures in the pipeline.

Methodology (2/5): Data Acquisition from SEC EDGAR

Step 2: Fetching the right filings at the right time

- Using the structured intent object, the constructDB node fetches SEC filings via the edgartools library. For full-report queries, the two most recent 10-K and 10-Q filings are retrieved; for specific queries, filings are filtered by form type and date range.
- Each filing is chunked at the section level using edgartools' built-in chunked_document API. Chunks shorter than 40 characters or containing TableBlock markers are discarded to remove noise and boilerplate.
- Each retained chunk is wrapped in a LangChain Document with metadata: ticker, form type, filing date, section name, chunk ID, and source URL. This metadata flows through to the retrieval stage and powers source attribution in the final output.
- The corpus is embedded with OpenAI's text-embedding-ada-002 model and stored in an in-memory Chroma vector store scoped to the current query session. Metadata-filtered search is used at retrieval time to ensure workers only retrieve from their assigned filing type and section scope.
- The in-memory vector store is built fresh per query session. Persisting the Chroma store across sessions to eliminate redundant embedding computation is identified as a key item for future scope.

Methodology (3/5): Intent Routing & Orchestration

Step 3: Routing by intent and generating a structured section plan

After the vector store is populated, a conditional edge evaluates the intent field and routes execution to one of two sub-graphs. “Specific” queries go to a single-step retriever-response chain. “Full_report” queries go to an orchestrator-worker-synthesizer chain.

Specific Query Path

The retriever node issues a semantic search (k=5) with a metadata filter on form type. Retrieved chunks are passed to the writer LLM (gpt-oss-20b) with a prompt requiring citation in the format: Source: <form>, Section: <item>, Company: <ticker>.

Full Report Path

The orchestrator LLM (llama-3.1-8b) receives the user query and returns a structured Sections object — one Section per filing type — each with a name, description, retrieval query, generation goal, and filing filter. LangGraph’s Send API fans these out to parallel llm_call workers.

Pydantic Validation

Three Pydantic models enforce output schema at runtime: QueryDecompose (query parsing), Section and Sections (orchestrator plan). LangChain’s with_structured_output() wrapper binds these schemas to LLM calls, raising errors on invalid responses.

Each sub-query is independently optimized for its target filing type and section scope, ensuring retrieval precision. The number of parallel workers is determined at runtime by the number of sections the orchestrator produces.

Methodology (4/5): Parallel Retrieval & Reasoning

Step 4: Simultaneous multi-document retrieval and LLM-based analysis

- For full-report queries, LangGraph's Send primitive fans sections out to parallel llm_call workers. Each worker retrieves k=5 chunks filtered by both form type and section metadata, ensuring that 10-K workers never pull 10-Q data and vice versa.
- For each worker, the top-5 semantically relevant chunks are retrieved via cosine similarity over the embedding space. These chunks are passed to the writer LLM (gpt-oss-20b at temperature 0, served via Groq) as a structured prompt to generate a standalone analytical section.
- Each worker is instructed to produce a standalone section grounded in its retrieved context. The writer LLM uses temperature 0 for deterministic, factual outputs. The query decomposer and orchestrator use llama-3.1-8b-instant at temperature 0 and 0.1 respectively.
- Each parallel worker produces an independent section covering one filing type (10-K or 10-Q). The synthesizer node then collects all completed sections and joins them into a complete multi-section financial report for the user.
- All LLM calls are routed through Groq, providing low-latency inference. Running worker sections in parallel via the Send API reduces total latency compared to sequential processing — important when multiple large filings are in scope.

Methodology (5/5): Report Synthesis

Step 5: Assembling parallel worker outputs into a coherent report

- Once all parallel llm_call workers complete, the synthesizer node collects their outputs. For full-report queries, it joins all completed sections into a single structured financial report covering MD&A, risk factors, financial statements, and business factors.
- For specific queries, the pipeline follows a simpler retriever-response path: the retriever fetches k=5 chunks with a metadata filter on form type, then the writer LLM generates a direct answer with structured citations (Source: <form>, Section: <item>, Company: <ticker>).
- The full report includes sections covering: Revenue and Growth Drivers, Segment Performance, Cash Flow and Capital Allocation, Non-Operating Items, and Accounting/Audit Disclosures — drawn from 10-K and 10-Q filings respectively.
- Each section includes explicit source citations pointing to the filing type, section name, and company ticker — making the output fully auditable. Users can trace every insight back to its primary disclosure, reducing the risk of misinformed financial judgments.
- LangSmith was employed during development for debugging and traceability — allowing each pipeline stage, LLM call, and retrieval result to be inspected independently. The LangGraph graph model enforces deterministic execution order while enabling parallel worker fan-out.

Techniques & Tools Used

Techniques

- RAG (Retrieval-Augmented Generation)
- Dense Semantic Embeddings
- Query Decomposition
- Parallel Graph Execution
- LLM Prompt Engineering

Core Libraries

- LangChain — document wrapping, retrieval chains, structured output binding
- LangGraph — directed state graph with Send API for parallel fan-out
- ChromaDB — in-memory vector store (scoped per query session)
- edgartools — For data API

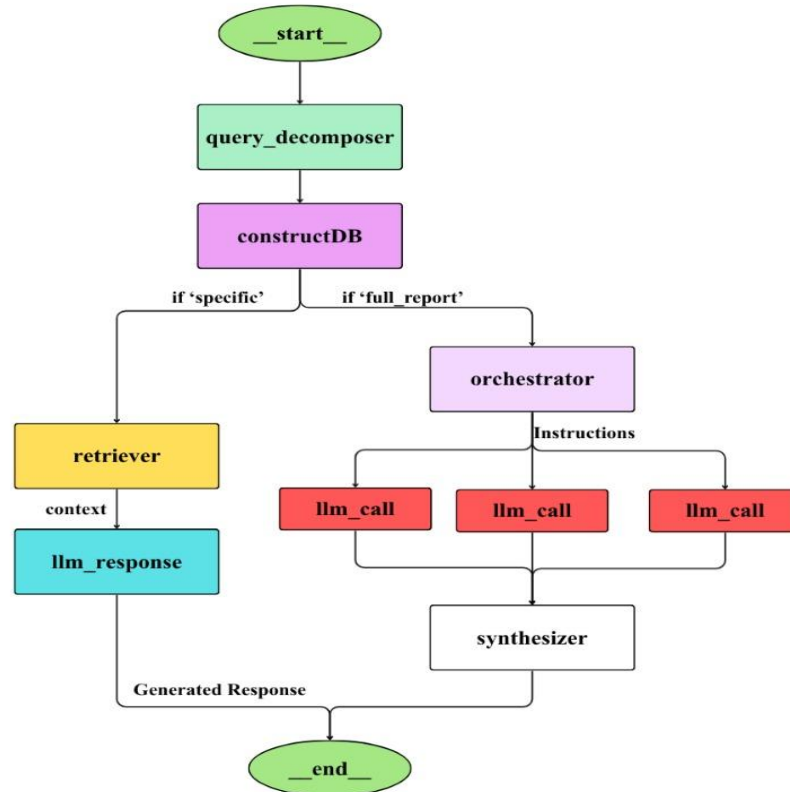
APIs & Services

- Groq Inference API — low-latency LLM serving (llama-3.1-8b-instant, gpt-oss-20b)
- OpenAI Embeddings API — text-embedding-ada-002 for vector encoding
- LangSmith — pipeline debugging, tracing, and step-level observability

Language & Infra

- Python 3.11 — primary language
- Pydantic — runtime schema validation for all structured LLM outputs
- Markdown + JSON — output formatting

System Architecture & Pipeline



Results & Output Structure

Example Query: “Provide a financial report on MSFT”

01

Specific Query Output

A direct natural-language answer grounded in retrieved SEC filing text, followed by supporting evidence. A References section cites Source: <form>, Section: <item>, Company: <ticker> for full transparency.

02

Full Report — 10-K Sections

Covers Revenue and Growth Drivers, Segment Performance (Productivity & Business Processes, Intelligent Cloud, More Personal Computing), and Accounting/Audit Disclosures drawn from the annual 10-K filing.

03

Full Report — 10-Q Sections

Covers recent quarterly MD&A, Cash Flow and Capital Allocation, Non-Operating Items, and Segment re-classification updates drawn from the most recent 10-Q quarterly filing.

04

Traceability & Auditability

Every section includes explicit citations (Source: <form>, Section: <item>, Company: <ticker>). LangSmith tracing was used during development to inspect each pipeline stage and verify retrieval quality end-to-end.

Observations: Why This Output Is Better

✓ Grounded, not hallucinated

Every claim in the output is traceable to a specific chunk from a real filing. Source citations include filing type, section name, and date — making the output fully auditable.

✓ Analytical depth, not extraction

The system doesn't just find where 'revenue' appears — it computes trends, compares time periods, and identifies divergences between what the 10-K projects and what the 10-Q shows.

✓ Cross-filing synthesis (10-K + 10-Q)

The full-report workflow synthesises independent sections from the 10-K and 10-Q into one coherent document — identifying how annual trends and quarterly performance relate to each other.

✓ Structured and reusable output

The structured, section-wise report is generated via the Streamlit-based front end and can be directly used for investment decisions, risk assessments, and strategic evaluations — all grounded in official SEC filings.

Comparison with Existing Systems

Capability	Standard RAG	FinQA System	ReAct Agent	Our System
Multi-filing type support	✗	✗ (10-K only)	Partial	✓ (10-K + 10-Q)
Query decomposition	✗	✗	✗	✓ (orchestrator + Send API)
Parallel retrieval	✗	✗	✗	✓ (concurrent nodes)
Structured output format	✗	Partial	✗	✓ (multi-section report)
Source citations	Partial	Partial	✗	✓ (form + section + ticker)
Cross-document synthesis	✗	✗	Partial	✓ (synthesizer node)
Real SEC EDGAR data	✗	✗ (fixed dataset)	✗	✓ (edgartools, live fetch)

Conclusion

- This project successfully demonstrates that a structured, orchestrated approach to financial document analysis produces significantly more useful outputs than standard single-step RAG or agent-based systems.
 - The implemented system correctly decomposes natural-language queries into typed parameters via Pydantic-validated LLM calls, retrieves filing-specific chunks using metadata-filtered RAG over an in-session ChromaDB store, and generates either targeted responses or structured multi-section reports depending on query intent.
 - The system is grounded in real, publicly available data from SEC EDGAR — not synthetic datasets or proprietary corpora — making it both transparent and reproducible.
 - The LangGraph graph model enforces deterministic execution order while enabling parallel worker fan-out via the Send API. Structured outputs backed by Pydantic validation prevent schema errors from propagating between pipeline stages — each stage can be inspected, replaced, or extended independently.
 - This work contributes a working proof-of-concept for orchestrated financial intelligence — a category of systems that goes beyond retrieval into structured, multi-document reasoning. It also lays the groundwork for practical applications in investment research, regulatory monitoring, and automated financial reporting.
-

Future Scope

Persistent Vector Store

Persist the Chroma store across sessions to eliminate redundant embedding computation for repeat queries on the same company and period — significantly reducing latency for follow-up analysis.

Comparison Agent

Implement a comparison agent enabling side-by-side analysis of two or more companies — a common analyst workflow. Each company would run its own retrieval pipeline with a final cross-company synthesis step.

Verification Node

Add a verification node that cross-checks numerical claims in generated sections against raw filing text — reducing the risk of unsupported figures appearing in reports and improving output reliability.

FastAPI Endpoint

Expose the pipeline via a FastAPI endpoint to allow integration with front-end tools and downstream applications — aligning the implementation with the full production architecture described in the project specification.

8-K Filing Support

Extend the pipeline to ingest 8-K event-triggered filings — covering material events such as leadership changes, acquisitions, and earnings surprises — completing the full three-filing analysis originally envisioned.

Team Contribution

Both team members contributed to bringing this project from idea to implementation:

Soham Dutta

Core System, Data Pipeline & LLM Integration

- Designed and implemented the full orchestration pipeline using LangGraph
- Built the query parsing module and prompt engineering for intent extraction
- Developed the SEC EDGAR data acquisition layer and document chunking pipeline
- Implemented embeddings (OpenAI text-embedding-ada-002) and in-memory ChromaDB vector store setup
- Implemented parallel llm_call workers via LangGraph Send API and Groq-hosted LLM inference (llama-3.1-8b-instant, gpt-oss-20b)
- Designed Pydantic schemas for structured LLM output validation and used LangSmith for pipeline debugging and traceability

Rajat Sharma

Testing, Documentation & Presentation

- Assisted in system integration, basic workflow setup, and testing different query cases across full-report and specific query modes
- Contributed to report writing, documentation, and overall project presentation flow
- Prepared presentation slides and explained the project workflow, system architecture, and results to the audience
- Prepared all presentation materials including this slide deck
- Reviewed literature and compiled the comparison analysis across related works
- Assisted in debugging retrieval quality and refining chunking parameters

Thank You

SEC Filing Intelligence System
B.Tech Final Year Project — 2024–25

Soham Dutta | Rajat Sharma

References: Lewis et al. (2020) · Chen et al. (2021) · Araci (2019) · Malo et al. · Reimers & Gurevych (2019) · Yao et al. (2023)